

MODULE 5.1 · Concept 3 of 7

Hybrid Planning Approaches

Combining classical structure with LLM flexibility — and recovering when plans fail

9 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

TH-000006

Prerequisite

AI-AG-IN-TH-000007

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHY HYBRID

Best of Both Worlds

Classical gives soundness. LLMs give language. Hybrid gives both.

Why Hybrid: Combining Strengths

Neither classical nor LLM planning alone covers all needs

Need	Classical	LLM	Hybrid Solution
Parse 'plan my vacation'	✗ Can't understand NL	✓ Native NL	LLM parses → classical structures
Guarantee valid ordering	✓ Precondition checking	✗ May misorderr	LLM generates → verifier checks
Handle surprises	✗ Closed world only	✓ Can adapt	LLM replans when execution fails
Track resource budgets	✓ Constraint propagation	✗ Loses track	Classical tracks budget, LLM plans within
Scale to complex goals	✗ Exponential search	✓ Prompt scales	LLM prunes options → classical searches reduced space

Pattern 1: LLM Generates → Classical Verifier Checks

The LLM proposes, the verifier validates



What the verifier checks:

- Step ordering: does step N have all inputs from previous steps?
- Budget constraints: does total cost stay within limits?
- Resource conflicts: are two steps using the same resource simultaneously?
- Feasibility: does each step use an available tool with valid parameters?

Pattern 2: Classical Structure → LLM Fills Details

The planner provides the skeleton, the LLM adds the flesh

Classical Skeleton:

1. SEARCH(transport, origin, dest)
2. COMPARE(results, criteria)
3. BOOK(selected_option)
4. CONFIRM(booking, user)

Generic template – works for flights, hotels, restaurants, etc.
Ordering guaranteed correct.

LLM Fills Details:

1. search_flights("BLR","GOA","Mar 20", max_price=5000, passengers=10)
2. "Compare by: price, then departure time, prefer non-stop"
3. book_flight(flight_id="6E-432", passengers=[list of 10 names])
4. "Send confirmation email to team@company.com with itinerary"

Classical structure ensures correct ordering. LLM fills in natural language parameters and context-specific details.

Pattern 3: Iterative Refinement — Plan → Execute → Replan

The most robust pattern: adapt the plan as reality unfolds



This is called 'plan repair'. The agent doesn't start over — it keeps completed steps and replans only the remaining ones.

Plan repair is more efficient than full replanning because:

- Completed steps are kept — no wasted work
- The agent has real data from executed steps — better informed replan
- Only remaining steps need to be regenerated
- Cost: 1 additional LLM call for replan vs. re-executing entire plan

Plan Repair: Worked Example

Flight cancelled mid-trip — agent replans from current state

Original Plan: BLR→MUM(fly) → MUM→GOA(train) → GOA→BLR(fly)

Step 1: ✓ Flew BLR→MUM successfully. Now in Mumbai.

Step 2: ✗ Train MUM→GOA cancelled due to track maintenance.

Plan Repair triggered. Current state: location=MUM, budget=₹10,800, days=2

Replanned Steps (replacing steps 2–3):

Step 2a: Search alternative MUM→GOA transport (bus, flight, cab)

Step 2b: Bus MUM→GOA found at ₹1,200 (vs train ₹800) — within budget

Step 3: Fly GOA→BLR as originally planned

Result: Trip completed. Extra cost: ₹400. Delay: 2 hours. No human intervention needed.

SECTION 2 — PRACTICAL

Verification, Tradeoffs, and Production Use

Making hybrid planning work in real agent systems

Verification Strategies

Three ways to check if a plan will actually work

Constraint Checking

Validate budget, timing, resource limits programmatically \$0 (code)

Always — first line of defense

Simulation / Dry Run

Execute plan in sandbox without real side effects

~\$0.01 (API calls to test endpoints)

When actions have real-world consequences (booking, sending)

LLM-as-Judge Review

Second LLM call critiques the plan for logical errors

~\$0.01 (one LLM call)

When ordering and completeness matter

Cost/Quality Tradeoffs: Hybrid vs. Pure LLM

Hybrid is more expensive per plan but saves on execution failures

Metric	Pure LLM Planning	Hybrid Planning	Difference
Plan generation cost	~\$0.03 (1 LLM call)	~\$0.05 (plan + verify)	+67% upfront
Execution failure rate	~20–30% (infeasible plans)	~5–10% (verified plans)	-60% failures
Cost of 1 failure	~\$0.15 (wasted execution)	~\$0.15 (same if it happens)	Same
Expected total cost	$\$0.03 + 25\% \times \$0.15 = \$0.07$	$\$0.05 + 7\% \times \$0.15 = \$0.06$	-15% total
Time to first result	Fast (no verification)	Slower (+1 LLM call)	~2 sec slower
Reliability	Lower	Higher	Key advantage

When to Use Hybrid: Decision Framework

Match planning strategy to task characteristics

Task Characteristic	Best Strategy	Why
Simple, low-stakes	Pure LLM (or just ReAct)	Verification overhead not worth it
Multi-step, user-visible	Plan-then-execute + LLM verify	User wants to approve plan before execution
Budget/resource constrained	Hybrid with constraint checker	Must track running totals
Dynamic environment	Interleaved + plan repair	Need to adapt as reality changes
Safety-critical actions	Hybrid + simulation + human gate	Must verify before irreversible actions

Default recommendation: start with pure LLM planning. Add hybrid verification only when failure cost justifies it.

Production Example: Devin Uses Hybrid Planning

The autonomous coding agent combines LLM plans with test-based verification

Devin's planning approach:

1. LLM reads the issue → generates a multi-file edit plan (which files, what changes, what order)
2. Plan verified against codebase: do the files exist? Are imports correct? Does the change break type checks?
3. Execute plan: write code changes file by file
4. Classical verification: run the test suite
5. If tests fail → plan repair: LLM reads error output, identifies the failure, replans the fix
6. Loop until all tests pass (typically 2–5 iterations)

Key insight: Devin's test suite IS the classical verifier. Running tests checks preconditions (code compiles), effects (tests pass), and constraints (no regressions). This is hybrid planning where the 'classical' component is the existing engineering infrastructure.

Key Takeaways

The essential points from this concept

- Three hybrid patterns:** LLM→verifier, classical skeleton→LLM details, iterative plan repair.
- Plan repair is the key differentiator** — adapt the plan when reality doesn't match expectations.
- Verification costs ~\$0.02 extra** but reduces execution failures by 60%.
- Match strategy to task:** simple→pure LLM, constrained→hybrid, safety-critical→full verification.
- Devin exemplifies hybrid planning:** LLM plans code changes, test suite verifies, replan on failure.

Next: AI-AG-IN-TH-000008 — Task Decomposition

AI-AG-IN-TH-000007 Complete

Hybrid Planning Approaches

Concept 3 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi